

面向边缘计算的 MD-SLAM 系统： 核心方法与实现细节

1 创新点 1: 高频视觉先验与曼哈顿结构豁免

1.1 详细方法与流程

针对边缘计算设备（如限制算力的移动端）在高度动态场景下的实时性衰减与特征误杀问题，本系统设计了基于双线程架构的级联概率滤波机制[cite: 1]。

- 高频视觉先验：** 在前端追踪线程中，部署边缘端优化的轻量级 YOLO 实例分割网络，利用边界框与掩码进行特征的极速初始筛查，提供高频视觉先验[cite: 1]。
- 级联概率滤波拒绝硬剔除：** 摒弃将动态边界框内特征全部直接“硬剔除”的传统做法，所有嫌疑特征将进入后续的级联概率滤波管线[cite: 1]。
- 曼哈顿结构豁免：** 在局部建图线程中，提取符合曼哈顿世界假设的正交平面[cite: 1]。若嫌疑特征点完美拟合这些正交结构（如墙壁、地板），算法将触发结构豁免机制，直接赋予其 $P_{dynamic} = 0$ 的动态概率，强制保留这些被动态物体遮挡或干扰的静态背景特征[cite: 1]。

1.2 数学建模

曼哈顿正交平面在三维空间中被数学公式化提取，表达为：

$$n^T P + d = 0$$

[cite: 1] 定义曼哈顿结构掩码函数 $M_{manhattan}$ ，当特征点属于该平面时，直接绕过极线几何校验，分配动态概率：

$$P_{dynamic}(P_i | M_{manhattan}(P_i) = 1) = 0$$

[cite: 1]

1.3 算法伪代码

```
1 # Input: Features {F}, Edge-YOLO Mask, Depth Map
2 # Output: Classified Features with Prior Probability
3
4 static_features = []
5 suspect_features = []
6
7 # Extract orthogonal planes based on Manhattan World assumption
8 manhattan_planes = extract_orthogonal_planes(depth_map)
```

```

9
10 for pt in F:
11     # High-frequency visual prior screening
12     if YOLO_Mask[pt.u, pt.v] == STATIC:
13         pt.p_dynamic = 0.0
14         static_features.append(pt)
15     else:
16         # Structural Immunity Mechanism
17         is_immune = False
18         pt_3d = BackProject(pt.u, pt.v, depth)
19
20         for plane in manhattan_planes:
21             if abs(dot(plane.n, pt_3d) + plane.d) < THRESHOLD:
22                 is_immune = True
23                 break
24
25         if is_immune:
26             pt.p_dynamic = 0.0 # Bypassing epipolar checks
27             static_features.append(pt)
28         else:
29             suspect_features.append(pt)
30
31 return static_features, suspect_features

```

Listing 1: Visual Prior and Manhattan Immunity

2 创新点 2: 极线几何校验与平面阴影剔除

2.1 详细方法与流程

对于未获得曼哈顿结构豁免的其余特征点，系统将执行几何与光度层面的深度校验，以解决间接动态干扰（如平滑表面上的反射与阴影）造成的轨迹漂移[cite: 1]。

- **极线几何校验：** 针对非曼哈顿平面上的点，计算其极线几何误差（桑普森距离），以此推导出基于几何特征的动态概率 P_{geo} [cite: 1]。
- **平面阴影剔除：** 动态物体产生的移动阴影极易被误认为静态角点。系统将潜在的阴影区域通过数学公式投影至已提取的曼哈顿地面（Ground Plane）上，并为该区域内的特征点分配阴影先验概率，执行精准剔除[cite: 1]。

2.2 数学建模

极线几何校验依赖计算桑普森距离 (Sampson distance) 来推导 P_{geo} [cite: 1]。同时, 平面阴影剔除模型将阴影先验概率 P_{shadow} 与 P_{geo} 融合, 更新该点的最终动态概率 $P_{dynamic}$ 。

2.3 算法伪代码

```

1 # Input: Suspect Features {S}, Fundamental Matrix F, Ground Plane
2 # Output: Features with evaluated P_dynamic
3
4 valid_features = []
5 shadow_regions = project_shadows_to_plane(dynamic_objects, ground_plane)
6
7 for pt in S:
8     # 1. Epipolar Geometry Verification
9     sampson_dist = compute_sampson_distance(pt, F)
10    p_geo = derive_geometric_probability(sampson_dist)
11
12    # 2. Planar Shadow Rejection
13    p_shadow = 0.0
14    if is_inside_projected_shadow(pt, shadow_regions):
15        p_shadow = assign_prior_shadow_probability()
16
17    # Fuse probabilities
18    pt.p_dynamic = max(p_geo, p_shadow)
19
20    if pt.p_dynamic < DYNAMIC_THRESHOLD:
21        valid_features.append(pt)
22
23 return valid_features

```

Listing 2: Epipolar Verification and Shadow Rejection

3 创新点 3: 密度感知的自适应点线联合图优化

3.1 详细方法与流程

在极端纹理退化或动态目标过大导致特征被大规模剔除的场景下, 系统容易面临特征匮乏 (Feature Starvation) 与跟踪丢失[cite: 1]。后端设计了密度感知的自适应点线联合 Bundle Adjustment (BA) 来解决此问题[cite: 1]。

- **特征密度评估:** 系统实时计算当前参与优化的活跃特征数量 N_{active} , 并定义特征密度惩罚系数 γ [cite: 1]。

- **自适应信息矩阵缩放:** 基于贝叶斯动态概率, 对优化图中的信息矩阵 (Information Matrices) 进行动态自适应缩放, 抑制高概率动态点的权重, 防止轨迹漂移[cite: 1]。
- **点线联合约束:** 当点特征出现严重退化或被施加高额惩罚时, 系统自动引入曼哈顿线特征 (Line Features) 提供鲁棒的方向约束, 实现联合优化兜底[cite: 1]。

3.2 数学建模

活跃特征数量记为 N_{active} , 对应密度惩罚系数为 γ [cite: 1]。后端自适应信息矩阵缩放的核心公式为:

$$\Omega_i^{new} = W_s \cdot \Omega_i$$

[cite: 1] 其中, 权重因子 W_s 由动态概率直接推导得出:

$$W_s = 1 - P_{dynamic}$$

[cite: 1]

3.3 算法伪代码

```

1 // Input: Valid Points {P}, Manhattan Lines {L}, Pose T
2 // Output: Optimized Pose T_opt
3
4 void DensityAwareOptimization(std::vector<MapPoint>& points, Matrix4d& T
5 ) {
6     Optimizer optimizer;
7     optimizer.addPoseVertex(T);
8
9     // 1. Feature Density Assessment
10    int N_active = calculate_active_features(points);
11    double gamma = compute_density_penalty_coefficient(N_active);
12
13    // 2. Adaptive Information Matrix Scaling
14    for (auto& pt : points) {
15        auto edge = new PointReprojectionEdge(pt);
16
17        // W_s = 1 - P_dynamic
18        double W_s = 1.0 - pt.p_dynamic;
19
20        // Omega_new = W_s * Omega_i
21        Matrix2d Omega_new = W_s * IdentityMatrix();
22        edge->setInformation(Omega_new);
23        optimizer.addEdge(edge);
24    }

```

```
24
25 // 3. Point-Line Joint Constraint Fallback
26 if (N_active < STARVATION_THRESHOLD) {
27     for (auto& line : manhattan_lines) {
28         auto edge_line = new LineProjectionEdge(line);
29         // Robust orientation constraints when point features are
30 degraded
31         edge_line->setInformation(IdentityMatrix() * HIGH_WEIGHT);
32         optimizer.addEdge(edge_line);
33     }
34 }
35 optimizer.optimize();
36 T = optimizer.getOptimizedPose();
37 }
```

Listing 3: Density-Aware Adaptive Point-Line BA